

MoneXa — Cahier des charges technique

Défi 2 — Hackathon ESIG Tech Arena 2026

Édition 2026 — 25 au 27 septembre

Stack : Django full + Flutter + IA multimodale

Plateforme de trésorerie pour PME ouest-africaines

3 rôles RBAC + audit immuable + chatbot TresorIA

Document officiel — Version 2.0

Table des matières

1. Présentation du projet MoneXa.....	1
2. Choix du Défi 2 et justification stratégique	1
2.1 Pourquoi le Défi 2 plutôt qu'un autre	2
2.2 Adéquation avec les compétences de l'équipe	2
2.3 Opportunité de marché et impact local.....	2
2.4 Risques assumés et mitigation.....	2
3. Objectifs du projet	3
3.1 Objectifs produit (MVP)	3
3.2 Objectifs pédagogiques.....	3
3.3 Objectifs business	3
3.4 Objectifs d'impact et de valeurs	3
4. Architecture générale	4
5. Stack technique et justification des choix	5
5.1 Pourquoi Django (vs FastAPI, Flask, Node.js)	5
5.2 Pourquoi Flutter (vs React Native, Kotlin/Swift natif)	5
5.3 Pourquoi PostgreSQL (vs MongoDB, MySQL, SQLite).....	6
5.4 Pourquoi un LLM multimodal (vs OCR + LLM texte)	6
5.5 Pourquoi Django Admin comme back-office (vs React admin custom)	7
5.6 Pourquoi services IA internes (vs microservices IA séparés)	7
5.7 Pourquoi un audit immuable via trigger PostgreSQL (vs soft delete)	8
6. Modèle de données.....	8
6.1 Les trois règles d'or financières en Django	9
6.2 Schéma relationnel textuel	9
7. Authentification et contrôle d'accès (RBAC)	10
7.1 Les trois rôles et leurs permissions.....	10
7.2 JWT pour Flutter, sessions pour Django Admin.....	10

7.3 2FA optionnel pour le Gérant	11
8. Backend Django — structure et composants.....	11
8.1 Organisation du projet en applications	11
8.2 Django Admin comme back-office.....	11
8.3 DRF API et viewsets	11
8.4 Management commands.....	12
8.5 Tests et qualité.....	12
9. Frontend web Django.....	12
9.1 Django Admin étendu	12
9.2 Templates Django + HTMX pour vues avancées.....	12
9.3 i18n et accessibilité.....	13
10. Application Flutter mobile.....	13
10.1 Architecture Clean Architecture + BLoC	13
10.2 Écrans principaux	13
10.3 Offline-first.....	13
10.4 State management BLoC	14
10.5 Notifications push.....	14
10.6 Sécurité et accessibilité	14
11. Pipeline IA — extraction LLM multimodale	14
11.1 Flux complet du pipeline	14
11.2 Prompt engineering et few-shots.....	15
11.3 Validation Pydantic et traçabilité.....	15
11.4 Fallback et plan B	15
12. Chatbot TresorIA — IA générative sécurisée	15
12.1 Principe — KPIs pré-calculés, jamais de SQL	16
12.2 Endpoints et sécurité	16
12.3 Cas d'usage typiques.....	16
13. Prévisions et détection d'anomalies	16
13.1 Prévisions Holt-Winters à J+7 et J+30	16
13.2 Détection d'anomalies hybride.....	17

13.3 Cascade de matching automatique	17
14. Intégrations IA additionnelles	17
14.1 Traduction automatique multilingue.....	18
14.2 Voice-to-text pour saisie mains-libres	18
14.3 Catégorisation intelligente des dépenses	18
14.4 Résumés hebdomadaires LLM (CFO virtuel)	18
14.5 Génération de factures PDF personnalisées	18
14.6 Autres intégrations IA prévues.....	19
15. Contrat d'API REST	19
16. Plan de développement 48 heures	20
16.1 Jour 1 — vendredi 25 septembre 2026 (H0 → H24).....	20
16.2 Jour 2 — samedi 26 septembre 2026 (H24 → H48)	21
16.3 Jour 3 — dimanche 27 septembre 2026 (H48 → démo)	21
17. Livrables et critères d'évaluation.....	23
17.1 Livrables attendus.....	23
17.2 Grille d'évaluation du jury.....	24
17.3 Conseils pour la démonstration.....	24
18. Axes d'amélioration à implémenter (roadmap)	25
18.1 Tier 1 — Court terme (1 à 3 mois post-hackathon).....	25
18.2 Tier 2 — Moyen terme (3 à 6 mois).....	25
18.3 Tier 3 — Long terme (6 à 18 mois, vision produit)	26
19. Annexes.....	27
19.1 FAQ	27
19.2 Lexique	27
19.3 Contacts	28

Note : cette table des matières est générée par champ. Pour mettre à jour les numéros de page après modification, faire un clic droit sur la table des matières puis « Mettre à jour les champs ».

1. Présentation du projet MoneXa

MoneXa est une plateforme de gestion intelligente de trésorerie conçue pour les PME ouest-africaines qui opèrent avec des paiements mobile money (T-Money, Moov Money, Flooz) au quotidien. Le projet est porté par une équipe participante au Défi 2 du hackathon ESIG Tech Arena 2026, dans une approche produit complète : backend Django, back-office Django Admin, frontend web Django, et application mobile native Flutter, avec intégration de plusieurs modèles d'intelligence artificielle pour automatiser les tâches répétitives et offrir une assistance en langage naturel aux utilisateurs.

La problématique visée est bien réelle : la majorité des petites entreprises ouest-africaines reçoivent leurs paiements via SMS mobile money, sans réconciliation automatique possible avec leurs factures émises. La comptabilité se fait à la main, les doublons passent inaperçus, les fraudes internes sont difficiles à détecter, et la trésorerie à 30 jours relève du pifomètre. MoneXa propose une réponse à cette problématique en s'appuyant sur trois piliers : l'automatisation par IA (extraction des SMS, matching automatique, détection d'anomalies), la rigueur financière (PostgreSQL, contraintes ACID, audit immuable), et l'accessibilité mobile (Flutter, offline-first, multilingue).

Ce cahier des charges est le document de référence de l'équipe pour la conception, le développement et la présentation du projet pendant les 48 heures du hackathon. Il sert également de support pour la démonstration devant le jury, qui y trouvera la justification des choix techniques opérés, la profondeur fonctionnelle visée, et la roadmap d'amélioration à implémenter au-delà de l'événement. L'équipe s'est engagée à suivre une démarche d'ingénierie explicite, sans raccourcis opaques, et à produire un livrable démontrable de bout en bout.

Pitch en une phrase

MoneXa est un CFO virtuel pour PME ouest-africaines : il photographie vos reçus mobile money, l'IA extrait et réconcilie automatiquement les paiements avec vos factures, vous alerte sur les anomalies, et vous donne une vision claire de votre trésorerie à 30 jours — accessible depuis votre téléphone, sans connexion permanente.

2. Choix du Défi 2 et justification stratégique

L'équipe a fait le choix stratégique de s'engager sur le Défi 2 (Application Mobile) plutôt que sur les quatre autres défis proposés par l'ESIG Tech Arena 2026. Ce choix n'est pas anodin : il engage toute l'équipe pendant 48 heures sur une stack technique précise, expose à des risques spécifiques, et conditionne les opportunités de partenariats industriels. Le présent chapitre explicite les raisons du choix et la manière dont l'équipe s'est positionnée par rapport aux autres défis.

2.1 Pourquoi le Défi 2 plutôt qu'un autre

Le Défi 2 a été choisi pour trois raisons principales. Premièrement, l'équipe dispose de compétences solides en développement mobile (Flutter, Dart) et en architecture backend Python (Django, DRF), ce qui constitue un avantage compétitif immédiat. Deuxièmement, le contexte ouest-africain du projet MoneXa impose une expérience mobile de qualité : les gérants et caissiers visés travaillent depuis leur téléphone, dans des conditions de connectivité variables, et attendent une application robuste, rapide et utilisable hors-ligne. Troisièmement, le Défi 2 valorise explicitement l'expérience utilisateur native, ce qui correspond à l'ambition produit de MoneXa : une plateforme agréable à utiliser au quotidien, et non un simple back-office de comptabilité.

2.2 Adéquation avec les compétences de l'équipe

La composition de l'équipe a été déterminante. Les membres ont une expérience pratique de Flutter en production (deux membres ont déjà déployé des apps sur les stores), de Django en contexte académique et professionnel (trois membres développent en Django depuis deux ans), et de Python pour l'IA (deux membres suivent un parcours Data/ML). Cette combinaison de compétences aligne parfaitement le projet avec les forces de l'équipe, et permet d'aborder sereinement les points techniques les plus risqués : le pipeline IA multimodal, l'offline-first sous Flutter, et la sécurisation des données financières.

2.3 Opportunité de marché et impact local

Le marché visé est porteur. L'Afrique de l'Ouest compte plus de 100 millions d'utilisateurs de mobile money, avec un volume annuel de transactions supérieur à 200 milliards d'euros. Une proportion significative de ces transactions provient de PME qui n'ont pas accès à des outils de gestion de trésorerie adaptés, faute de solutions locales, abordables et en français. MoneXa vise à combler ce manque, en s'appuyant sur une connaissance fine du contexte (opérateurs locaux T-Money et Moov, formats de SMS spécifiques, langues locales Ewé et Kabyé, calendrier SYSCOHADA). L'impact local est double : démocratiser l'accès à des outils de gestion financière pour les PME, et participer à la formalisation progressive de l'économie ouest-africaine.

2.4 Risques assumés et mitigation

Le choix du Défi 2 expose l'équipe à des risques assumés et identifiés à l'avance. Le premier risque est la complexité du déploiement Flutter en 48 heures (build APK, signatures, configurations). La mitigation : utiliser Expo EAS ou un pipeline fastlane pré-configuré, et prévoir une démo sur appareil physique dès H30 pour valider le build. Le deuxième risque est l'accessibilité de l'API LLM multimodale pendant le hackathon : indisponibilité réseau, latence élevée, coût dépassant le quota. La mitigation : pré-cacher 20 vrais SMS traités à l'avance pour la démo, et prévoir un pipeline de secours PaddleOCR + LLM texte. Le troisième risque est l'intégration Django + Flutter (CORS, JWT,

offline sync), qui peut consommer du temps précieux. La mitigation : déployer le premier « smoke test » dès H6 pour détecter les problèmes d'intégration le plus tôt possible.

3. Objectifs du projet

3.1 Objectifs produit (MVP)

Le périmètre produit minimum viable (MVP) à livrer pour la fin du hackathon comprend : la gestion des utilisateurs avec trois rôles distincts (Gérant, Comptable, Caissier), le CRUD des factures avec numérotation automatique, l'upload d'images de SMS ou reçus mobile money via l'app mobile avec extraction IA et matching automatique, le tableau de bord de trésorerie avec KPIs agrégés par canal, la file de validation des paiements A_VALIDER pour le comptable, et un journal d'audit immuable accessible au gérant. Ce périmètre, s'il est livré de bout en bout, constitue une démonstration convaincante de la valeur produit MoneXa.

3.2 Objectifs pédagogiques

Au-delà du produit, le projet vise des objectifs d'apprentissage pour l'équipe. Chaque membre s'est engagé sur un axe de montée en compétence : architecture Django monolithe avec services internes, intégration LLM multimodal en production (prompt engineering, validation Pydantic, gestion des erreurs), architecture offline-first sous Flutter (Hive, sync différée, gestion de conflits), et intégration sécurité (RBAC, audit immuable, secrets management). Le hackathon est pour l'équipe l'occasion de confronter ces apprentissages à un cas réel, avec la pression du délai et l'exigence de qualité.

3.3 Objectifs business

MoneXa est pensé comme un projet au-delà du hackathon. Les objectifs business sont triples : établir un contact avec les partenaires industriels présents pour explorer des suites (stages, incubation, financement), valider l'intérêt du produit auprès de PME pilotes prêt à tester une version beta, et préparer une éventuelle levée de pré-amorçage auprès d'investisseurs locaux ouest-africains intéressés par la fintech. La qualité du livrable de démonstration et la clarté de la roadmap d'amélioration sont essentielles à cette ambition business.

3.4 Objectifs d'impact et de valeurs

L'équipe s'engage sur des valeurs fortes : transparence de l'IA (chaque décision IA est tracée, le raw_text et le score de confiance sont conservés), inclusion linguistique (interface disponible en français et dans deux langues locales au minimum), accessibilité numérique (TalkBack/VoiceOver supportés, contrastes validés), sécurité by design (jamais de secret en clair, 2FA pour le gérant, audit immuable), et impact local mesurable (réduction du temps de réconciliation comptable pour les

PME cibles). Ces valeurs ne sont pas des slogans : elles se traduisent par des choix techniques concrets détaillés dans les chapitres suivants.

Tableau 1 — Synthèse des objectifs MoneXa

Axe	Objectif	Indicateur de succès
Produit	MVP fonctionnel déployé et démontrable	Démo live 5 min sans incident
Pédagogique	4 axes de montée en compétence	Retour d'expérience documenté
Business	3 partenaires identifiés au village	Cartes de visite échangées
Impact local	Beta-testeurs PME intéressés	5 intentions d'essai post-hackathon
Valeurs — IA	Traçabilité totale des décisions IA	100% des paiements avec raw_text + confidence
Valeurs — inclusion	FR + 2 langues locales	Ewé + Kabyé sur l'app mobile
Valeurs — sécurité	2FA + audit immuable	Trigger PostgreSQL actif en prod

4. Architecture générale

L'architecture retenue repose sur un monolithe Django bien structuré, qui combine en un seul déploiement backend l'API REST (DRF), l'ORM (PostgreSQL), le back-office (Django Admin), et le frontend web pour les rôles comptables et gérants (templates Django + HTMX). L'application mobile native est développée séparément en Flutter, et consomme l'API REST du backend. Les services d'intelligence artificielle sont implémentés en interne dans le backend Django, sous forme de modules Python appelés par les endpoints API, sans microservice séparé. Ce choix d'architecture est volontairement simple, et traduit une discipline d'ingénierie : éviter la sur-ingénierie en 48 heures.

Le flux principal de données est le suivant : (1) le caissier saisit une facture ou photographie un reçu de paiement mobile money depuis l'app Flutter ; (2) l'image est envoyée à l'endpoint POST /api/payments/evidence/ du backend Django ; (3) le pipeline IA interne extrait les données via un LLM multimodal, valide le JSON via Pydantic, crée un objet Payment avec raw_text et ai_confidence, et le soumet au matcher ; (4) le matcher exécute la cascade de réconciliation (référence exacte → montant + fenêtre 7 jours → fuzzy nom/téléphone → A_VALIDER) et met à jour le statut ; (5) les données sont stockées dans PostgreSQL, une entrée d'audit est créée, et le comptable est notifié ; (6) le caissier et le comptable consultent le dashboard depuis respectivement l'app mobile et le back-office web.

3 décisions structurantes dès H0

1. Custom User AVANT la première migration (règle n°1 de Django) — sinon refonte d'authentification douloureuse. 2. Mode API pur DRF, pas de templates Django mélangés — mais Django Admin conservé comme back-office. 3. Services IA en interne (pas de microservice séparé) — inutile en 48h, et le déployer serait un gaspillage de temps précieux.

5. Stack technique et justification des choix

Le présent chapitre justifie chaque choix technique majeur opéré par l'équipe, en présentant les alternatives envisagées et la raison pour laquelle elles n'ont pas été retenues. Cette justification est essentielle à la fois pour la cohérence interne de l'équipe pendant le hackathon, et pour la démonstration devant le jury, qui valorise les démarches d'ingénierie explicites.

5.1 Pourquoi Django (vs FastAPI, Flask, Node.js)

Django 5 a été retenu comme framework backend principal. Les alternatives envisagées étaient FastAPI (Python, plus léger et async natif), Flask (Python, minimaliste), et Node.js avec Express ou Nest.js (JavaScript/TypeScript). Django l'emporte sur quatre critères décisifs. Premièrement, l'Admin Django intégré : il offre un back-office complet et gratuit pour les rôles Comptable et Gérant, ce qui évite de développer un frontend web admin custom et libère environ 8 heures de développement. Deuxièmement, l'ORM mature avec contraintes et migrations versionnées : l'intégrité financière (DecimalField, unique=True, transaction.atomic) est démontrée par construction, pas affirmée. Troisièmement, la documentation auto-générée via drf-spectacular (Swagger UI) qui constitue un argument produit pour le jury. Quatrièmement, la cohérence Python avec l'IA : le backend et les services IA partagent le même langage, sans friction d'équipe.

FastAPI aurait été plus performant en termes de latence et plus léger, mais aurait nécessité de construire l'Admin à la main (au moins une journée de travail), et n'apporterait pas le Swagger auto. Flask aurait été trop minimaliste pour un projet de cette envergure. Node.js aurait introduit une friction de langage avec l'IA (Python) et n'aurait pas permis à l'équipe Data d'aider le backend sans réapprendre un environnement. Django est donc le choix le plus pragmatique pour un hackathon de 48 heures.

5.2 Pourquoi Flutter (vs React Native, Kotlin/Swift natif)

Flutter 3.22+ a été retenu pour le développement de l'application mobile. Les alternatives envisagées étaient React Native (JavaScript/TypeScript), Kotlin natif Android + Swift natif iOS (deux codebases séparés), et Ionic (web hybride). Flutter l'emporte sur trois critères. Premièrement, la performance native : Flutter compile en code natif via Skia, offrant une fluidité d'animation et de transition très supérieure à React Native (qui passe par un pont JavaScript). Pour une app fintech utilisée quotidiennement, cette fluidité est essentielle à l'adoption. Deuxièmement, la productivité

: un seul codebase Dart pour Android et iOS, avec un hot reload exceptionnel qui accélère le développement. Troisièmement, la cohérence visuelle : Flutter offre le même rendu sur toutes les plateformes, sans les variations Material You/San Francisco qui surviennent avec React Native.

React Native aurait permis de réutiliser les compétences JavaScript de l'équipe, mais la friction du pont JS pour les apps à forte interaction (uploads, transitions, charts) aurait pénalisé l'UX. Le natif Kotlin+Swift aurait été trop coûteux en temps pour couvrir deux plateformes en 48 heures. Ionic aurait compromis la fluidité et l'accessibilité native. Flutter est donc le meilleur compromis pour le projet.

5.3 Pourquoi PostgreSQL (vs MongoDB, MySQL, SQLite)

PostgreSQL 16 a été retenu comme base de données principale. Les alternatives envisagées étaient MongoDB (NoSQL document), MySQL (SGBD relationnel classique), et SQLite (fichier local). PostgreSQL l'emporte sur quatre critères. Premièrement, la rigueur ACID native et le support des transactions qui est non-négociable pour des données financières. Deuxièmement, le type DecimalField précis (jamais d'erreur d'arrondi comme avec les floats), avec contraintes UNIQUE pour anti-doublon natif. Troisièmement, le support des triggers SQL qui permet d'implémenter un journal d'audit immuable au niveau base de données — un argument décisif pour le jury, qui démontre la maturité de la démarche. Quatrièmement, la richesse de l'écosystème (extensions, types JSONB, index avancés) qui ouvre des axes d'amélioration à moindre coût.

MongoDB aurait été plus flexible pour des données non structurées (raw_text, métadonnées IA), mais aurait compromis l'intégrité transactionnelle et rendu l'audit immuable plus difficile à implémenter. MySQL aurait été acceptable mais moins riche en extensions et avec un écosystème Python moins mature (psycopg vs mysqlclient). SQLite aurait été parfait pour un prototype, mais inadapté à un déploiement cloud multi-utilisateur.

5.4 Pourquoi un LLM multimodal (vs OCR + LLM texte)

L'extraction des données des SMS et reçus mobile money est confiée à un LLM multimodal (GPT-4o-mini Vision ou Gemini Flash), qui analyse directement l'image sans passer par un OCR préalable. Les alternatives envisagées étaient un pipeline PaddleOCR + LLM texte, ou un OCR Tesseract + LLM texte, ou un OCR pur sans LLM. Le LLM multimodal l'emporte sur trois critères. Premièrement, la robustesse aux photos floues et mal cadrées, qui sont la norme pour les photos de SMS prises par les caissiers sur le terrain. Deuxièmement, l'élimination d'une étape OCR qui peut échouer (PaddleOCR casse souvent en Docker, Tesseract est peu tolérant). Troisièmement, la tolérance aux variations de format des SMS entre opérateurs (T-Money vs Moov vs Flooz), que le LLM gère nativement via le few-shot prompting.

Le pipeline OCR + LLM texte est conservé comme option de secours (Plan B) au cas où l'API LLM multimodale serait indisponible ou trop coûteuse. Cette double option sera mentionnée au pitch sous le libellé « NLP + vision par ordinateur », conformément au règlement du défi. Le coût d'un

appel LLM multimodal est d'environ 0,002 USD pour 20 000 tokens, soit un budget négligeable pour un hackathon, mais qui devient un paramètre à surveiller en production (stratégie de cache et de batching à prévoir).

5.5 Pourquoi Django Admin comme back-office (vs React admin custom)

Le back-office pour les rôles Comptable et Gérant est implémenté via Django Admin, étendu et personnalisé plutôt que de développer un frontend admin custom (React admin, Refine, React-Admin). Les alternatives envisagées étaient un frontend admin custom en React (Refine ou React-Admin) consommant l'API DRF, ou un template Django + HTMX pour les vues comptables avancées. Django Admin l'emporte sur trois critères. Premièrement, le gain de temps : l'Admin offre CRUD, recherche, filtres, pagination, et permissions en quelques lignes de code, ce qui libère facilement 6 à 8 heures de développement. Deuxièmement, la robustesse : l'Admin est maintenu par la core team Django, sécurisé, et éprouvé en production depuis 20 ans. Troisièmement, le plan de secours : si le frontend web custom ou l'app mobile flanche à H40, l'Admin peut porter une partie de la démo de secours (en français, avec `LANGUAGE_CODE='fr'`).

Un frontend admin custom en React aurait permis une UX plus fine, mais au prix d'un temps de développement non négligeable. Pour le MVP, l'Admin Django suffit largement, et peut être enrichi post-hackathon si le produit passe en phase beta. Le template Django + HTMX est conservé pour les vues comptables avancées (rapports, exports SYSCOHADA), qui ne sont pas couvertes nativement par l'Admin.

5.6 Pourquoi services IA internes (vs microservices IA séparés)

Les services d'intelligence artificielle (extraction LLM, chatbot, prévisions, anomalies) sont implémentés en interne dans le backend Django, sous forme de modules Python du package `finance/services/`, et non comme des microservices séparés (FastAPI IA, conteneurs Docker distincts). Les alternatives envisagées étaient un microservice FastAPI dédié à l'IA, ou une architecture serverless (Lambda + API Gateway). Le choix de l'intégration interne s'impose sur trois critères. Premièrement, la simplicité de déploiement en 48 heures : un seul processus, un seul langage, une seule source de vérité. Deuxièmement, la cohérence Python : backend et IA partagent les modèles Django, les helpers, et les utilitaires (audit, etc.). Troisièmement, l'absence de besoin en scalabilité horizontale à court terme : le volume d'appels IA en démo est faible, et le coût d'un microservice serait disproportionné.

Un microservice FastAPI dédié à l'IA aurait permis une scalabilité indépendante et une dérogation de langage (mais Python reste le meilleur choix pour l'IA), mais aurait coûté au moins 4 heures de configuration et de déploiement, sans bénéfice tangible en 48 heures. L'architecture serverless aurait introduit une dépendance cloud (AWS Lambda) et une complexité de tests qui ne se justifie pas pour un MVP.

5.7 Pourquoi un audit immuable via trigger PostgreSQL (vs soft delete)

Le journal d'audit des opérations sensibles est implémenté via un mécanisme de **hash-chaining cryptographique**, inspiré du principe fondamental des blockchains : chaque entrée d'audit contient le hash SHA-256 de l'entrée précédente en plus de ses propres données, formant une chaîne vérifiable. Toute modification a posteriori d'une entrée — même par un accès direct à la base de données — brise la chaîne et est détectable instantanément via un endpoint de vérification d'intégrité.

Les alternatives envisagées étaient un trigger PostgreSQL bloquant les UPDATE/DELETE, un soft delete, ou un audit applicatif via signals Django. Le hash-chaining l'emporte sur trois critères. Premièrement, la **détection active de falsification** : contrairement au trigger qui *empêche* la modification, la chaîne de hash permet de *prouver* qu'aucune modification n'a eu lieu, ce qui est un argument plus fort et plus proche du principe blockchain. Deuxièmement, la **portabilité et la simplicité** : le mécanisme est implémenté en Python pur, sans dépendance à une fonctionnalité spécifique du SGBD, ce qui facilite les tests et la démonstration. Troisièmement, la **valeur pédagogique et démonstrative** : le jury peut assister en direct à une tentative de falsification et à sa détection immédiate, ce qui constitue une preuve tangible d'immuabilité plutôt qu'une affirmation.

Tableau 2 — Synthèse comparative des choix techniques

Composant	Choix retenu	Alternative écartée	Raison principale
Backend	Django 5 + DRF	FastAPI / Flask / Node	Admin gratuit + ORM mature + IA même langage
Mobile	Flutter 3.22+	React Native / Kotlin / Swift	Performance native + 1 codebase + UX fluide
BDD	PostgreSQL 16	MongoDB / MySQL / SQLite	ACID + DecimalField + triggers audit
IA extraction	LLM multimodal	PaddleOCR + LLM texte	Robustesse photos floues + 1 appel API
Back-office	Django Admin	React-Admin custom	Gain 6-8h + robustesse + plan B démo
Services IA	Internes Django	Microservice FastAPI	Simplicité 48h + cohérence Python
Audit	Hash-chaining (SHA-256)	Soft delete / signals	Détection de falsification vérifiable, portable, démontrable en live

6. Modèle de données

Le modèle de données MoneXa est structuré autour de cinq entités principales : User (compte utilisateur avec rôle), Account (compte de paiement par canal), Invoice (facture émise), Payment

(paiement reçu avec traçabilité IA), et AuditLog (journal d'audit immuable). Une sixième entité Expense est ajoutée pour couvrir les flux sortants (fournisseurs), conformément à l'exigence du règlement du défi qui demande la vision unifiée du fonds de roulement.

6.1 Les trois règles d'or financières en Django

Trois règles non négociables s'appliquent à toute manipulation de données financières dans MoneXa. Première règle : jamais de FloatField pour l'argent. Le type FloatField est un nombre flottant qui produit des erreurs d'arrondi ($0,1 + 0,2 \neq 0,3$). Pour l'argent, on utilise systématiquement DecimalField(max_digits=14, decimal_places=2), qui garantit une précision exacte au centime près. Deuxième règle : les contraintes DB sont de l'anti-fraude natif. La contrainte unique=True sur le champ provider_ref d'un Payment rend physiquement impossible la création d'un doublon de paiement avec la même référence opérateur, même en cas de bug applicatif. Troisième règle : toute écriture de solde ou de paiement est encapsulée dans transaction.atomic() pour garantir la cohérence en cas d'erreur (si l'enregistrement d'un paiement échoue à mi-chemin, toute la transaction est annulée).

6.2 Schéma relationnel textuel

Le modèle User hérite de AbstractUser (custom user, voir règle n°1 Django). Il ajoute un champ role avec trois valeurs possibles (GERANT, COMPTABLE, CAISSIER) et un champ phone pour le numéro de téléphone mobile money de l'utilisateur. La table Account représente les comptes de paiement par canal (T-Money, Moov, Flooz, Banque, Espèces), avec un nom personnalisable et un type channel. La table Invoice stocke les factures émises, avec référence unique auto-générée (FACT-2026-XXXX), montant en DecimalField, statut (EN_ATTENTE, RECONCILIE, ANOMALIE, ANNULE), et lien vers l'utilisateur émetteur.

La table Payment est l'entité centrale du pipeline IA. Elle stocke le montant, la référence opérateur (provider_ref, unique=True), le canal, les informations du payeur (nom, téléphone), la date de paiement, l'image de preuve (evidence_image), le texte brut extrait par l'IA (raw_text, pour traçabilité), le score de confiance de l'IA (ai_confidence), la méthode de matching (AUTO_REF, AUTO_MONTANT, MANUEL, IA), le statut (NON_RATTACHE, A_VALIDER, RECONCILIE, ANOMALIE), et les liens vers la facture rapprochée et l'utilisateur créateur. Des index sont créés sur paid_at et channel pour optimiser les requêtes du dashboard.

La table Expense est l'ajout majeur par rapport au playbook initial : elle couvre les flux sortants (paiements aux fournisseurs, salaires, loyer, etc.), avec fournisseur, catégorie, montant, justificatif OCR, et date. Cette entité est essentielle pour répondre à l'exigence du règlement du défi qui demande une vision unifiée du fonds de roulement. La table AuditLog stocke chaque opération sensible, avec utilisateur, action, entité, ID d'entité, détails JSON, et adresse IP. Elle est protégée par un trigger PostgreSQL qui empêche toute modification ou suppression.

Bon réflexe dashboard — solde à la volée

Ne stockez pas les soldes 'à l'aveugle' dans la base. Calculez-les à la volée par agrégation : `Payment.objects.filter(channel='TMONEY').aggregate(Sum('amount'))`. Jamais de désynchronisation entre le solde stocké et la réalité des paiements, et c'est une seule requête SQL performante grâce aux index sur `paid_at` et `channel`.

7. Authentification et contrôle d'accès (RBAC)

7.1 Les trois rôles et leurs permissions

MoneXa implémente un contrôle d'accès basé sur les rôles (RBAC) à trois niveaux : Gérant, Comptable, et Caissier. Chaque rôle dispose d'un périmètre d'actions précis, défini dans la matrice ci-dessous. Le Caissier est l'utilisateur de terrain : il crée les factures, upload les preuves de paiement, et consulte son dashboard personnel. Il n'a pas accès aux données des autres caissiers, ni aux fonctions de validation. Le Comptable est l'opérateur de back-office : il consulte l'ensemble des paiements, valide les paiements A_VALIDER, exporte les rapports, et corrige les données de démo. Le Gérant est l'administrateur : il a toutes les permissions du Comptable, plus la résolution des anomalies, l'accès au journal d'audit, et la gestion des utilisateurs.

Tableau 3 — Matrice RBAC MoneXa

Action	Caissier	Comptable	Gérant
Créer une facture	✓	✓	✓
Uploader une preuve de paiement	✓	✓	✓
Consulter ses paiements	✓ (les siens)	✓ (tous)	✓ (tous)
Valider un paiement A_VALIDER	—	✓	✓
Exporter rapports (Excel/PDF)	—	✓	✓
Résoudre une anomalie	—	—	✓
Consulter le journal d'audit	—	—	✓
Gérer les utilisateurs	—	—	✓
Accès Django Admin back-office	—	✓ (lecture)	✓ (full)

7.2 JWT pour Flutter, sessions pour Django Admin

L'authentification est gérée de manière hybride. L'application Flutter mobile utilise des JSON Web Tokens (JWT) via `simplejwt` : l'utilisateur obtient un access token (validité 15 minutes) et un refresh token (validité 7 jours) en échange de ses identifiants. Les tokens sont stockés dans le

flutter_secure_storage, jamais en clair. Le backend Django Admin utilise les sessions Django classiques avec cookies sécurisés (HttpOnly, Secure, SameSite=Lax), qui ne sont pas compatibles avec une app mobile mais conviennent parfaitement à un back-office web.

7.3 2FA optionnel pour le Gérant

Le rôle Gérant dispose de privilèges étendus (résolution d'anomalies, gestion utilisateurs, accès audit), ce qui justifie une protection renforcée. MoneXa propose une authentification à deux facteurs (2FA) optionnelle pour ce rôle, via le package django-otp. Le 2FA se déclenche à la connexion : après saisie du mot de passe, un code TOTP est demandé (généré par une app comme Google Authenticator ou Authy). Le 2FA est activé à la demande du gérant depuis son profil, et n'est pas imposé par défaut pour ne pas pénaliser l'expérience de démo. En production, il serait imposé pour tous les rôles ayant accès à l'audit et à la gestion des utilisateurs.

8. Backend Django — structure et composants

8.1 Organisation du projet en applications

Le projet Django est organisé en cinq applications distinctes, chacune avec une responsabilité unique. L'application accounts gère le custom User, l'authentification JWT, les permissions RBAC, et les profils utilisateurs. L'application finance contient les modèles Account, Invoice, Payment, Expense, et les services IA (ai_pipeline, matcher, anomalies, forecast). L'application auditing contient le modèle AuditLog, le helper d'écriture, et la migration qui installe le trigger PostgreSQL. L'application reporting contient les KPIs du dashboard, les exports Excel et PDF, et le chatbot TresorIA. L'application assistant est dédiée au chatbot en langage naturel. Cette structure garantit que chaque app a une responsabilité claire, et qu'un nouveau développeur peut s'orienter rapidement.

8.2 Django Admin comme back-office

Le Django Admin est personnalisé pour offrir un back-office complet et utilisable par les comptables et gérants. Les Personnalisation incluent : list_display riche pour chaque modèle (montant, canal, statut, payeur, date), list_filter sur canal et statut, search_fields sur provider_ref et payer_name, et actions personnalisées en masse (marquer_valide, marquer_anomalie, exporter_excel). L'Admin est en français (LANGUAGE_CODE='fr'), avec un TIME_ZONE='Africa/Lome' pour les horodatages. Les permissions sont gérées via is_staff et les groupes Django : les comptables appartiennent au groupe 'comptable' qui a accès en lecture à toutes les données de finance, les gérants au groupe 'gerant' qui a toutes les permissions.

8.3 DRF API et viewsets

L'API REST est exposée via Django REST Framework avec des ViewSets et des sérialiseurs par ressource. Les permissions sont gérées au niveau des ViewSets via une classe de base RolePermission qui vérifie le rôle de l'utilisateur courant. Les endpoints sont routés via DefaultRouter, et la documentation Swagger est auto-générée par drf-spectacular à l'URL /api/schema/swagger-ui/. Cette documentation est un argument produit fort pour le jury, qui peut explorer l'API en 2 minutes et comprendre sa structure complète.

8.4 Management commands

Trois management commands sont prévues : (1) seed_demo pour générer un jeu de données de démonstration reproductible (3 mois d'historique, 200 paiements, 3 anomalies typées, 50 factures) — cette commande est essentielle pour la démo et les tests ; (2) anonymize_data pour anonymiser les données réelles en vue d'un usage en démo (remplacement des noms et téléphones par des données fictives) ; (3) generate_forecast pour pré-calculer les prévisions Holt-Winters à J+7 et J+30 et les cacher en base (optimisation des performances du dashboard).

8.5 Tests et qualité

Les tests sont implémentés avec pytest-django. Le périmètre de test prioritaire est le pipeline IA (20 vrais SMS en fixtures pour valider le parser LLM), la cascade de matching (tests unitaires par branche), les permissions RBAC (tests d'intégration par rôle), et l'audit immuable (test que toute tentative de modification du journal lève une erreur). La couverture indicative visée est de 40% minimum, valorisée par le jury comme preuve de démarche d'ingénierie sérieuse.

9. Frontend web Django

9.1 Django Admin étendu

Le frontend web pour les rôles Comptable et Gérant est implémenté via Django Admin étendu, plutôt que par un frontend custom. Les personnalisations incluent : surcharge des templates Admin pour adapter le branding (logo MoneXa, couleurs cyan, footer personnalisé), ajout de vues personnalisées pour les rapports SYSCOHADA et les exports Excel, intégration de HTMX pour des interactions en place sans recharge de page (filtres dynamiques, modals de validation, etc.), et intégration de TailwindCSS via django-tailwind pour styliser les vues custom. Cette approche permet d'offrir une expérience back-office de qualité sans développer un frontend complet, et libère du temps pour l'app mobile et le pipeline IA.

9.2 Templates Django + HTMX pour vues avancées

Les vues comptables avancées qui ne sont pas couvertes par l'Admin (rapports personnalisés, exports SYSCOHADA, dashboard avec graphiques, chatbot TresorIA) sont implémentées en templates Django + HTMX. HTMX permet de faire des interactions AJAX déclaratives (hx-get, hx-

post, hx-swap) sans écrire de JavaScript, ce qui accélère le développement et facilite la maintenance. Les graphiques du dashboard sont rendus côté serveur via Chart.js ou ApexCharts (rendered via Django template tags), avec mise à jour dynamique via HTMX polling.

9.3 i18n et accessibilité

Le frontend web Django est internationalisé via le framework i18n natif de Django. Le français est la langue par défaut (LANGUAGE_CODE='fr'), avec traductions en Ewé et Kabyé pour les libellés principaux. Les templates respectent les bonnes pratiques d'accessibilité : attributs ARIA, balises sémantiques HTML5, contrastes validés WCAG AA, navigation clavier complète. Le frontend web est également responsive (media queries CSS) pour permettre une consultation depuis un smartphone en cas de besoin.

10. Application Flutter mobile

10.1 Architecture Clean Architecture + BLoC

L'application Flutter suit une architecture en couches (Clean Architecture) avec le pattern BLoC (Business Logic Component) pour la gestion d'état. La séparation des responsabilités est la suivante : la couche Présentation (widgets + BLoC) gère l'UI et la logique de présentation ; la couche Domain (Use Cases + Entities) contient la logique métier pure ; la couche Data (Repositories + Data Sources) gère les appels API et le cache local. Cette architecture garantit une testabilité maximale (chaque couche est testable indépendamment) et une évolutivité (les changements de source de données n'impactent pas la présentation).

10.2 Écrans principaux

L'app Flutter comprend cinq écrans principaux : (1) l'écran de connexion avec authentification JWT et 2FA optionnel pour le gérant ; (2) l'écran d'accueil (dashboard) avec KPIs agrégés, répartition par canal, et dernières opérations ; (3) l'écran d'upload de preuve qui déclenche le pipeline IA et affiche un loader animé « Analyse IA en cours... » ; (4) l'écran de liste des paiements avec filtres (par statut, par canal, par date) et tri ; (5) l'écran de profil avec réglages, bascule 2FA, changement de langue, et déconnexion. Une bottom navigation bar permet de basculer entre les écrans, et chaque écran supporte le pull-to-refresh pour rafraîchir les données.

10.3 Offline-first

L'app est conçue en mode offline-first, ce qui est essentiel dans le contexte ouest-africain où la connectivité est intermittente. Le cache local est géré via Hive (NoSQL léger embarqué dans Flutter), qui stocke les dernières données consultées (paiements, factures, KPIs). Quand l'utilisateur consulte une donnée, l'app affiche d'abord le cache local (instantané), puis fait un appel API en arrière-plan pour rafraîchir. Les actions de l'utilisateur (création de paiement, upload de preuve) sont stockées

en file d'attente locale si l'appareil est hors-ligne, et synchronisées au retour réseau. Les conflits éventuels sont gérés par un timestamp server-side qui prend le pas sur le timestamp local.

10.4 State management BLoC

Le state management est assuré par le pattern BLoC (Business Logic Component), qui sépare la logique métier de la présentation. Chaque écran dispose de son propre BLoC, qui émet des états en réponse à des événements (UserLoaded, PaymentUploaded, AnomalyDetected, etc.). Les widgets écoutent les états du BLoC via BlocBuilder et réagissent en conséquence. Cette architecture permet une testabilité unitaire de la logique et une séparation claire des responsabilités. Pour les états globaux (authentification, langue), un BLoC global est utilisé au niveau de l'application.

10.5 Notifications push

Les notifications push sont implémentées via Firebase Cloud Messaging (FCM), avec un canal dédié 'anomalies' (priorité haute, sonnerie) et un canal 'info' (priorité basse, silencieux). Les notifications sont déclenchées côté serveur par les signals Django lorsqu'une anomalie est détectée ou qu'un paiement A_VALIDER est en attente depuis plus de 24 heures. Le token FCM est enregistré côté serveur lors de la première connexion de l'utilisateur.

10.6 Sécurité et accessibilité

La sécurité de l'app mobile repose sur quatre couches : (1) flutter_secure_storage pour les tokens JWT (jamais en clair dans SharedPreferences) ; (2) authentification biométrique optionnelle (Face ID / fingerprint) pour l'ouverture de l'app ; (3) certificate pinning pour les appels API (protection contre le MITM) ; (4) obfuscation ProGuard pour le build release (protection contre le reverse engineering). L'accessibilité est assurée par le support de TalkBack (Android) et VoiceOver (iOS), avec des sémantiques sémantiques (Semantics widget) sur tous les éléments interactifs, et des contrastes validés WCAG AA.

11. Pipeline IA — extraction LLM multimodale

11.1 Flux complet du pipeline

Le pipeline IA d'extraction est le composant différenciant de MoneXa. Le flux est le suivant : (1) l'utilisateur upload une image de SMS ou reçu mobile money depuis l'app Flutter ; (2) l'endpoint POST /api/payments/evidence/ reçoit l'image et la stocke temporairement ; (3) le service finance/services/ai_pipeline.py appelle un LLM multimodal (GPT-4o-mini Vision ou Gemini Flash) avec un prompt structuré et des few-shots ; (4) la réponse JSON est validée par Pydantic (montant, référence, opérateur, émetteur, date) — si la validation échoue, une exception est levée et l'utilisateur est invité à saisir manuellement ; (5) un objet Payment est créé avec raw_text (texte

brut extrait), ai_confidence (score de confiance), et evidence_image (image d'origine) ; (6) le matcher est appelé en cascade pour tenter la réconciliation automatique.

11.2 Prompt engineering et few-shots

Le prompt envoyé au LLM multimodal est soigneusement conçu pour maximiser la précision d'extraction. Il contient : (a) une instruction système qui précise le rôle ('Tu es un assistant d'extraction de données de reçus mobile money ouest-africains'), (b) un schéma JSON attendu (montant, référence, opérateur, type d'opération, émetteur, date), (c) une règle absolue ('N'invente jamais. Si une information n'est pas visible, retourne null pour ce champ.'), et (d) des few-shots (exemples) pour T-Money, Moov, et Flooz, qui couvrent les variations de format entre opérateurs. Le prompt est testé sur 20 vrais SMS anonymisés avant le hackathon pour valider la robustesse.

11.3 Validation Pydantic et traçabilité

La validation du JSON extrait par le LLM est confiée à Pydantic, qui définit un schéma strict (PaymentExtraction) avec des types précis (float pour montant, str pour référence, etc.) et des contraintes (montant > 0, référence au format attendu). En cas d'échec de validation, une exception ValueError est levée, et le paiement n'est pas créé — l'utilisateur est invité à saisir manuellement les informations. Cette validation rigoureuse garantit que seules des données exploitables entrent en base, et la traçabilité (raw_text + ai_confidence) permet au jury et au comptable de comprendre chaque décision IA.

11.4 Fallback et plan B

Un fallback de secours est prévu au cas où l'API LLM multimodal serait indisponible ou trop coûteuse : le pipeline PaddleOCR + LLM texte. PaddleOCR extrait le texte de l'image, puis un LLM texte (GPT-4o-mini ou Mistral) parse le texte selon le même schéma JSON. Ce pipeline est plus fragile (PaddleOCR peut échouer sur les images floues) et plus lent (deux appels au lieu d'un), mais constitue une sauvegarde acceptable. L'équipe mentionnera explicitement au pitch que MoneXa supporte les deux pipelines ('NLP + vision par ordinateur'), conformément au libellé du défi.

Coût et latence — budget IA

Un appel LLM multimodal (GPT-4o-mini Vision) coûte environ 0,002 USD pour 20 000 tokens, et prend 2 à 5 secondes. Pour un hackathon de démo, le coût est négligeable. En production, avec 1000 paiements/jour par PME cliente, le budget IA serait d'environ 2 USD/jour/PME — acceptable, mais à surveiller. La stratégie de cache (prompts identiques) et de batching (regroupement d'images) sont des axes d'optimisation à implémenter en phase de croissance.

12. Chatbot TresorIA — IA générative sécurisée

TresorIA est un chatbot intelligent qui permet aux utilisateurs de poser des questions en langage naturel sur leur trésorerie, et d'obtenir des réponses chiffrées instantanées. Il coche la case 'IA générative' du règlement du défi, et constitue le moment wow numéro deux du pitch (après l'upload de SMS). Sa conception est volontairement sécurisée : le LLM ne génère jamais de SQL, et ne fait que formuler en langage naturel des KPIs pré-calculés par le backend.

12.1 Principe — KPIs pré-calculés, jamais de SQL

Le principe fondamental de TresorIA est le suivant : le backend pré-calcule environ 15 KPIs (solde par canal, encaissé 7 jours, factures en retard, prévision J+30, top 5 clients, etc.) et les injecte dans le prompt du LLM. L'utilisateur pose sa question en langage naturel (« Combien ai-je encaissé en T-Money cette semaine ? »), le LLM reçoit la question + les KPIs en contexte, et répond en langage naturel (« Cette semaine, vous avez encaissé 350 000 FCFA via T-Money, dont 120 000 FCFA pour la facture FACT-2026-0047. »). Le LLM ne génère jamais de SQL, n'a jamais accès à la base de données, et ne peut pas produire de données inventées puisqu'il ne dispose que des KPIs fournis.

12.2 Endpoints et sécurité

Le chatbot est exposé via l'endpoint POST /api/assistant/ask/, qui prend en entrée la question de l'utilisateur et retourne la réponse en langage naturel. La sécurité est assurée par : (a) authentification JWT obligatoire ; (b) rate limiting via DRF Throttling (10 questions/minute par utilisateur pour éviter l'abus) ; (c) validation de la question par un filtre de contenu (rejet des questions non liées à la trésorerie) ; (d) journalisation de chaque question et réponse dans AuditLog pour traçabilité. Le LLM utilisé est GPT-4o-mini (rapide et abordable) avec un timeout de 10 secondes ; en cas de timeout, l'utilisateur est invité à reformuler.

12.3 Cas d'usage typiques

Les cas d'usage typiques de TresorIA couvrent quatre familles de questions. Les questions de solde (« Combien ai-je sur T-Money ? », « Quel est mon solde total ? ») sont les plus fréquentes et doivent être traitées en moins de 3 secondes. Les questions d'historique (« Quelles factures sont en retard ? », « Qui sont mes 5 meilleurs clients ? ») permettent à l'utilisateur d'explorer ses données sans naviguer dans l'UI. Les questions de prévision (« Quelle est ma trésorerie prévue à J+30 ? ») valorisent le module forecast. Les questions métier (« Ai-je des anomalies à résoudre ? ») orientent l'utilisateur vers les actions à mener.

13. Prévisions et détection d'anomalies

13.1 Prévisions Holt-Winters à J+7 et J+30

Le module de prévisions utilise la méthode Holt-Winters (lissage exponentiel triple) appliquée aux flux nets quotidiens (encaissements - décaissements) sur 90 jours d'historique. Cette méthode

capture la tendance, la saisonnalité, et le niveau, et produit des prévisions à court terme (J+7) et moyen terme (J+30) avec un intervalle de confiance à 80%. L'implémentation utilise statsmodels (ExponentialSmoothing), et le code fait environ 40 lignes. Les prévisions sont recalculées quotidiennement via la management command `generate_forecast`, et cachées en base pour éviter le recalcul à chaque consultation du dashboard.

13.2 Détection d'anomalies hybride

La détection d'anomalies est hybride, combinant des règles métier prioritaires et un scoring ML non supervisé. Les règles métier couvrent les cas évidents : doublon de `provider_ref` (détecté par contrainte unique PostgreSQL), écart facture/paiement (montant payé différent du montant facturé), paiement sans facture (paiement reçu mais non rattaché à une facture), et paiement hors fenêtre temporelle (date de paiement antérieure à la facture). Ces règles sont évaluées en temps réel à chaque création de paiement, et les anomalies détectées sont stockées avec un niveau de gravité (info, warning, critical).

Le scoring ML utilise l'algorithme Isolation Forest de scikit-learn, entraîné sur 90 jours d'historique de paiements. Il identifie les paiements atypiques (montant inhabituel, horaire atypique, canal inhabituel pour un client donné) qui ne sont pas détectés par les règles métier. Le score d'anomalie (entre 0 et 1) est stocké avec chaque paiement, et les paiements au score $> 0,7$ sont automatiquement flaggés comme anomalies potentielles pour validation humaine. Cette approche hybride (« règles + ML non supervisé ») est présentée au jury comme un compromis pragmatique entre explicabilité (règles) et détection fine (ML).

13.3 Cascade de matching automatique

La réconciliation automatique des paiements avec les factures suit une cascade à quatre niveaux : (1) référence exacte — si le `provider_ref` du paiement contient la référence de la facture (FACT-2026-XXXX), le matching est automatique et le statut passe à RECONCILIE ; (2) montant + fenêtre 7 jours — si le montant correspond à une facture impayée et que la date de paiement est dans la fenêtre de 7 jours après l'émission, le matching est automatique mais le statut reste A_VALIDER le temps de la validation comptable ; (3) fuzzy nom/téléphone — si le nom du payeur ou son téléphone correspondent (similarité $> 0,8$) à un client connu avec facture impayée, le matching est proposé mais reste A_VALIDER ; (4) si aucune correspondance n'est trouvée, le paiement reste NON_RATTACHE et remonte en anomalie.

14. Intégrations IA additionnelles

Au-delà du pipeline d'extraction et du chatbot TresorIA, MoneXa intègre plusieurs modules d'IA qui visent à faciliter les tâches des utilisateurs et à enrichir l'expérience produit. Ces modules ne sont pas tous prioritaires pour le MVP du hackathon, mais sont spécifiés et, pour certains, implémentés en MVP léger pour démontrer la richesse de la vision produit.

14.1 Traduction automatique multilingue

L'app mobile propose une traduction automatique de l'interface en trois langues : français (par défaut), Ewé, et Kabyé. Les traductions des libellés statiques sont réalisées en amont par un traducteur humain (qualité garantie). Pour les contenus dynamiques (messages d'erreur, réponses du chatbot), une traduction automatique via l'API DeepL ou Google Translate est proposée à l'utilisateur via un bouton 'Traduire'. Cette fonctionnalité est essentielle pour l'inclusion linguistique des PME rurales ouest-africaines, et constitue un argument d'impact fort face au jury.

14.2 Voice-to-text pour saisie mains-libres

L'app mobile intègre la reconnaissance vocale (speech-to-text) via l'API native Flutter (speech_recognition package), qui permet au caissier de dicter le nom d'un client ou le montant d'une facture plutôt que de les saisir manuellement. Cette fonctionnalité est utile dans deux cas : (a) caissier en mobilité, mains occupées ; (b) caissier peu à l'aise avec la saisie textuelle. La reconnaissance vocale supporte le français, et peut être étendue à l'Ewé et au Kabyé via un modèle Whisper fine-tuné (axe d'amélioration à moyen terme).

14.3 Catégorisation intelligente des dépenses

Le module Expense (flux sortants) intègre une catégorisation automatique des dépenses par LLM : lorsque l'utilisateur saisit une dépense (fournisseur, montant, libellé), le backend appelle un LLM léger (GPT-4o-mini ou Mistral) qui propose une catégorie parmi une liste prédéfinie (loyer, salaires, fournitures, transport, etc.). L'utilisateur peut confirmer ou modifier la catégorie proposée. Cette catégorisation automatique facilite l'analyse des dépenses par catégorie dans le dashboard, et constitue une fonctionnalité produit différenciante.

14.4 Résumés hebdomadaires LLM (CFO virtuel)

Chaque lundi, un résumé hebdomadaire est généré automatiquement par LLM à partir des KPIs de la semaine écoulée, et envoyé au gérant par email et via l'app mobile. Le résumé suit une structure fixe : chiffres clés (encaissé, décaissé, solde net), faits marquants (anomalies détectées, paiements en retard), et recommandations (actions à mener dans la semaine). Cette fonctionnalité incarne le slogan produit de MoneXa (« votre CFO virtuel vous écrit tous les lundis »), et constitue une valeur produit forte au-delà du hackathon.

14.5 Génération de factures PDF personnalisées

MoneXa génère automatiquement des factures PDF à partir des données saisies par le caissier, avec un design professionnel (logo, en-tête, pied de page, QR code de paiement pour T-Money/Moov). Le QR code est généré côté backend via la librairie qrcode, et contient les informations de paiement (référence, montant, opérateur). Le PDF est généré via ReportLab ou

WeasyPrint, et stocké en Cloudinary pour accès partagé. Cette fonctionnalité complète la boucle facturation et permet à la PME d'émettre des factures professionnelles sans solution dédiée.

14.6 Autres intégrations IA prévues

- Détection de tentatives de fraude : scoring de risque par client (ratio paiements à l'heure / retards / annulations), avec alerte en cas de scoring anormal.
- Suggestions de relance client : LLM qui propose un texte de relance personnalisé pour les factures impayées > 7 jours, avec ton adapté (commercial, ferme, etc.).
- OCR pour anciens relevés : module complémentaire pour importer des relevés bancaires scannés (6 derniers mois) et créer l'historique initial lors de l'onboarding d'une nouvelle PME cliente.
- Prédiction de saisonnalité : à partir de 12 mois d'historique, détection des périodes de creux et de pics, avec alertes proactives.
- Smart taggage de factures : proposition automatique de tags pour les factures (projet, client, campagne), basée sur le contenu du libellé.

15. Contrat d'API REST

L'API REST de MoneXa est exposée via Django REST Framework, documentée automatiquement par drf-spectacular à l'URL `/api/schema/swagger-ui/`. Tous les endpoints nécessitent une authentification JWT (sauf `/api/auth/token/` qui est public), et appliquent les permissions RBAC. Le rate limiting est fixé à 60 requêtes/minute par utilisateur authentifié, et à 10 requêtes/minute pour l'endpoint du chatbot (limite anti-abus). La pagination est fixée à 20 résultats par page sur les endpoints de liste, avec paramètres `?page` et `?page_size` personnalisables.

Tableau 4 — Endpoints de l'API MoneXa

Métho de	Route	Rôle min.	Description
POST	<code>/api/auth/token/</code>	public	JWT login (access + refresh)
POST	<code>/api/auth/refresh/</code>	public	Renouvellement du token
GET	<code>/api/auth/me/</code>	connecté	Profil utilisateur + rôle
GET/POST	<code>/api/invoices/</code>	caissier+	CRUD factures (num. auto)
GET/PATCH	<code>/api/invoices/{id}/</code>	comptable+	Détail / validation facture
POST	<code>/api/payments/evidence/</code>	caissier+	Upload image → pipeline IA → matching

Méthode	Route	Rôle min.	Description
POST	/api/payments/manual-text/	caissier+	Fallback : texte SMS collé
GET	/api/payments/?status=A_VALIDER	comptable+	File de validation
PATCH	/api/payments/{id}/validate/	comptable+	Décision humaine
GET	/api/accounts/	comptable+	Comptes + soldes agrégés
GET	/api/dashboard/summary/	connecté	KPIs + répartition par canal
GET	/api/reports/forecast/?days=30	comptable+	Prévision trésorerie J+7 / J+30
GET	/api/anomalies/	gérant	File d'anomalies
GET	/api/audit-logs/	gérant	Journal immuable (lecture seule)
POST	/api/assistant/ask/	connecté	Chatbot TresorIA
GET	/api/reports/export/?format=xlsx	comptable+	Export Excel / PDF / SYSCOHADA

La documentation Swagger auto-générée est un argument produit fort pour le jury : elle démontre que l'équipe a pensé l'API comme un produit, et non comme un simple backend. Le lien </api/schema/swagger-ui/> sera mis en avant lors du pitch comme preuve de professionnalisme.

16. Plan de développement 48 heures

Le plan de développement sur 48 heures est structuré en deux jours, avec des jalons clairs à chaque étape. Le plan est conçu pour absorber les imprévus : si un blocage survient, l'équipe sait quelles fonctionnalités peuvent être sacrifiées en premier (le PWA, les notifications WhatsApp) sans compromettre la démonstration.

16.1 Jour 1 — vendredi 25 septembre 2026 (H0 → H24)

Tableau 5 — Planning détaillé Jour 1

Créneau	Mission	Jalon de validation
18h00	Cérémonie d'ouverture, badgeage, installation équipe	—
18h30	Diffusion du sujet Open Innovation (informatif)	—
19h00	Top départ, setup repo Git, Docker Compose, projet Django	Repo initial poussé

Créneau	Mission	Jalon de validation
19h00-20h30	Custom User + settings (env, CORS, JWT, TIME_ZONE=frica/Lome)	makemigrations OK
20h30-22h00	Modèles Account, Invoice, Payment, Expense + Admin Django	Admin accessible
22h00-00h00	DRF ViewSets, permissions RBAC, helper audit, trigger immuable	/api/invoices/ répond
00h00-02h00	Premier déploiement 'fumée' Render + Vercel + CORS JWT	URL publique back OK

16.2 Jour 2 — samedi 26 septembre 2026 (H24 → H48)

Tableau 6 — Planning détaillé Jour 2

Créneau	Mission	Jalon de validation
08h00-10h00	Petit-déjeuner + sync équipe, code review matinal	—
10h00-12h00	Pipeline IA : appel LLM multimodal, Pydantic, endpoint /evidence/	Premier paiement créé via IA
12h00-14h00	Déjeuner + tests sur 20 vrais SMS anonymisés	Taux de réussite > 80%
14h00-16h00	Matcher + statuts, anomalies (règles), forecast Holt-Winters	Cascade complète OK
16h00-18h00	Flutter : login, dashboard KPIs, upload écran avec loader IA	App mobile démarre
18h00-20h00	Chatbot TresorIA (KPIs pré-calculés injectés)	Question / réponse fonctionnelle
20h00-22h00	Dinner + seed_demo (3 mois données + 3 anomalies), exports Excel	Données de démo prêtes
22h00-00h00	Tests E2E des 3 rôles, corrections bugs critiques	Parcours complet validé

16.3 Jour 3 — dimanche 27 septembre 2026 (H48 → démo)

Tableau 7 — Planning détaillé Jour 3

Créneau	Mission	Jalon de validation
08h00-10h00	Finalisations (README, screenshots, Swagger UI vérifié)	README complet
10h00-12h00	Wire de la démo live (facture 50 000 FCFA préparée)	Scénario de démo verrouillé

Créneau	Mission	Jalon de validation
12h00-14h00	Vidéo de secours 60s, gel du code, env de démo Docker local prêt	Plan A et Plan B prêts
14h00-15h00	Répétitions pitch chronométrées (5 min max)	Pitch tient dans le temps
15h00	Arrêt officiel des développements	—
15h30	Sessions de démonstrations (par équipe)	—
16h30	Délibération du jury	—
18h00	Cérémonie de clôture + remise des prix	—

Créneau	Mission	Jalon de validation
Plan B de démo — anticipation des risques Si la démo live échoue (panne réseau, latence LLM, indisponibilité backend), l'équipe dispose de trois sauvegardes : (1) une vidéo de secours de 60 secondes démontrant le parcours principal ; (2) une démo depuis le Django Admin en back-office, qui fonctionne hors-ligne ; (3) un seed_demo local Docker qui rejoue le scénario sur le portable. Le jury est informé du plan B et valorise cette prévoyance comme preuve de maturité professionnelle.		

17. Livrables et critères d'évaluation

17.1 Livrables attendus

Les livrables attendus à la fin des 48 heures sont les suivants : (1) un dépôt Git public contenant le code backend Django et l'app Flutter, l'historique des commits visible, et un README détaillé (architecture, schéma, lancement, screenshots, comptes de test par rôle) ; (2) une API backend déployée sur Render ou Railway, accessible publiquement avec Swagger UI en vitrine ; (3) une app Flutter mobile distribuée en APK debug et/ou via Expo EAS, installable sur un appareil physique fourni par l'équipe ; (4) une management command `seed_demo` rejouable générant le jeu de données de démonstration ; (5) une démonstration live de 5 minutes avec scénario verrouillé (upload d'un SMS de paiement 50 000 FCFA → extraction IA → matching → dashboard) ; (6) une vidéo de secours de 60 secondes.

17.2 Grille d'évaluation du jury

Le jury évalue l'équipe sur sept critères pondérés, ramenés à 100 points. La note finale est la moyenne des notes attribuées par les membres du jury, en retirant la note maximale et la note minimale pour limiter les effets extrêmes.

Tableau 8 — Grille d'évaluation MoneXa

Critère	Pondération	Application à MoneXa
Innovation et créativité	15 %	Pipeline LLM multimodal + chatbot sécurisé + résumés hebdo LLM
Technicité	20 %	Django monolithe + Flutter offline-first + IA interne + triggers PostgreSQL
Qualité du code	15 %	Tests pytest + structure apps + README + Swagger auto + audit immuable
UX / UI	10 %	Flutter animé + Admin personnalisé + i18n FR/Ewe/Kabiyé
Complétude fonctionnelle	15 %	RBAC + factures + paiements + IA + chatbot + exports + audit
Présentation orale	15 %	Scénario 5 min + vidéo secours + Q&A jury préparé
Esprit d'équipe	10 %	Répartition par axe + daily sync + plan B partagé

17.3 Conseils pour la démonstration

La démonstration de 5 minutes est structurée en trois temps : (a) 30 secondes de pitch problématique/solution ('MoneXa est un CFO virtuel pour PME ouest-africaines...') ; (b) 3 minutes 30 de démonstration live guidée par un seul membre (login caissier → upload SMS → analyse IA → matching → vue comptable → question chatbot) ; (c) 1 minute de conclusion et perspectives (axes d'amélioration à implémenter). Un membre reste au clavier pour montrer du code si le jury le demande. La vidéo de secours est prête à être lancée en 10 secondes en cas d'échec de la démo live.

18. Axes d'amélioration à implémenter (roadmap)

Ce chapitre liste les axes d'amélioration à implémenter au-delà du hackathon, structurés en trois tiers par horizon temporel. L'objectif est de montrer au jury que l'équipe a une vision produit long terme, et que les choix techniques opérés ne ferment aucune porte mais ouvrent au contraire un champ d'évolution cohérent.

18.1 Tier 1 — Court terme (1 à 3 mois post-hackathon)

Les axes court terme visent à fiabiliser et industrialiser le MVP livré au hackathon, en vue d'une phase beta avec des PME pilotes. Ces axes ne modifient pas l'architecture, mais comblent les trous laissés volontairement par le hackathon (sécurité, monitoring, automatisation).

- Connecteurs agrégateurs de paiement (CinetPay, Flutterwave) pour ingestion automatique des SMS mobile money, sans nécessité d'upload manuel — remplace la photo de SMS par un flux temps réel.
- Import de relevés bancaires CSV + rapprochement automatique avec les paiements mobile money (vue unifiée trésorerie multi-canaux).
- Celery + Redis pour les traitements batch (pré-calcul des prévisions, génération des résumés hebdo, envoi des notifications WhatsApp) sans surcharger l'API synchrone.
- Sentry monitoring des erreurs en production + Grafana dashboard de supervision (requêtes/min, latence LLM, taux de succès matching IA) — argument 'nous monitorons la prod'.
- Tests E2E Playwright sur le back-office web + tests d'intégration Flutter integration_test sur l'app mobile (couverture indicative 60%).
- 2FA imposé pour tous les rôles (et non plus seulement optionnel pour le Gérant) en production.
- Conformité RGPD renforcée : politique de rétention des données (raw_text de l'IA supprimé après 90 jours), droit à l'oubli, portabilité des données.

18.2 Tier 2 — Moyen terme (3 à 6 mois)

Les axes moyen terme enrichissent la valeur produit et préparent la monétisation. Ils introduisent des fonctionnalités différenciantes qui structurent l'offre MoneXa sur le marché ouest-africain.

- Module Dépenses & Fournisseurs complet (suivi des flux sortants, gestion des baux, salaires, fournisseurs récurrents) avec catégorisation IA.
- Exports SYSCOHADA complets (journal de caisse, journal de banque, balance, compte de résultat) au format OHADA, ancrage local massif.

- Fermeture de caisse espèces quotidienne avec contrôle anti-fraude (écarts flaggés automatiquement) — complète le volet 'fuites de capital'.
- Factures PDF avec QR code de paiement (lien T-Money/Moov) — boucle facturation complète et instantanée.
- Rappels automatiques des factures impayées > 7 jours, avec texte généré par LLM (ton adapté au client).
- Scoring de fiabilité clients (ratio paiements à l'heure / retards / annulations par client) — mini-IA prédictive pour la gestion du risque client.
- Rapport hebdomadaire LLM envoyé par email — le CFO virtuel qui écrit tous les lundis.
- App mobile Expo réutilisant l'API REST sans réécriture (même codebase Flutter, build mobile Expo en complément).
- PWA offline-first pour le caissier (contexte de connectivité ouest-africaine, argument d'impact fort).
- Notifications WhatsApp Business aux gérants sur anomalies critiques (canal préféré des PME ouest-africaines).

18.3 Tier 3 — Long terme (6 à 18 mois, vision produit)

Les axes long terme projettent MoneXa dans une vision de produit plateforme, avec des fonctionnalités qui dépassent la stricte trésorerie pour englober la gestion financière globale de la PME, et préparent une éventuelle levée de fonds.

- Multi-boutiques / multi-guichets par entreprise (gestion centralisée d'un réseau de points de vente).
- Apple Wallet / Google Wallet integration (cartes de fidélité client, reçus de paiement numérique).
- Marketplace de services financiers (assurance, crédit court terme) avec partenaires bancaires locaux.
- Prévision de saisonnalité avancée (12 mois d'historique), avec alertes proactives sur creux attendus.
- Détection de fraude avancée (scoring risque par client, analyse de patterns inhabituels via autoencodeurs).
- Module de paie (génération bulletins de paie, déclarations sociales automatisées au format OHADA).
- API publique (open banking) pour intégration par des tiers (ERP, logiciels comptables, courtiers).
- Mode multi-devise (FCFA, euro, dollar) pour les PME à l'international.

- IA générative avancée : assistant vocal multilingue (Ewé, Kabyé, Français), génération de rapports annuels, explications de variances.
- Internationalisation régionale : extension à d'autres pays ouest-africains (Sénégal, Côte d'Ivoire, Burkina Faso, Mali) avec adaptation des opérateurs mobile money locaux.

Vision long terme — MoneXa plateforme CFO virtuel

À 18 mois, MoneXa vise à devenir la plateforme de référence du CFO virtuel pour PME ouest-africaines, avec une base de 10 000 PME clientes, une marketplace de services financiers intégrée, une API publique pour l'écosystème fintech local, et une présence dans 5 pays ouest-africains. L'IA générative devient le socle de l'expérience utilisateur, avec un assistant vocal multilingue, des rapports automatisés, et une détection de fraude avancée. La levée de pré-amorçage visée est de 1,5 M€ pour accélérer la distribution commerciale et l'enrichissement IA.

19. Annexes

19.1 FAQ

- Puis-je participer sans équipe ? Non, le hackathon requiert des équipes de 3 à 5 membres. L'organisation peut compléter une équipe incomplète sur demande.
- Puis-je utiliser un LLM ? Oui, à condition de l'indiquer explicitement dans le README et lors de la démo, en précisant les usages. L'usage non déclaré est considéré comme de la fraude.
- Le code que je produis m'appartient-il ? Oui. L'ESIG et les partenaires obtiennent une licence non exclusive d'usage à des fins pédagogiques et de promotion. Les accords spécifiques aux datasets sont signés au cas par cas.
- Que se passe-t-il si mon déploiement tombe la veille de la démo ? Préparez toujours une vidéo de secours de 60 secondes. Le jury accepte la vidéo en remplacement de la démo live, avec une légère pénalité.
- Le backend Django doit-il être déployé en production pour la démo ? Oui, mais une démo locale Docker est acceptée comme plan B. Privilégiez le déploiement Render ou Railway pour la démo live.

19.2 Lexique

BLoC : Business Logic Component, pattern de state management pour Flutter qui sépare la logique métier de la présentation via des événements et des états.

Clean Architecture : architecture en couches (Présentation, Domain, Data) qui garantit la testabilité et l'évolutivité, en isolant la logique métier des sources de données.

DecimalField : type de champ Django qui stocke des nombres décimaux exacts (sans erreur d'arrondi), utilisé systématiquement pour les montants financiers.

DRF : Django REST Framework, extension de Django qui expose des API REST via des ViewSets et des sérialiseurs.

Holt-Winters : méthode de lissage exponentiel triple qui modélise tendance + saisonnalité + niveau, utilisée pour les prévisions à court et moyen terme.

Isolation Forest : algorithme de machine learning non supervisé qui détecte les observations atypiques dans un jeu de données, basé sur la construction d'arbres aléatoires.

LLM multimodal : Large Language Model capable de traiter à la fois le texte et les images, comme GPT-4o Vision ou Gemini Flash, utilisé par MoneXa pour l'extraction des reçus mobile money.

Offline-first : approche de développement mobile où l'app fonctionne d'abord à partir du cache local, et se synchronise avec le backend en arrière-plan, essentielle dans les contextes de connectivité intermittente.

RBAC : Role-Based Access Control, modèle de contrôle d'accès où les permissions sont accordées par rôle (Gérant, Comptable, Caissier) plutôt que par utilisateur individuel.

SYSCOHADA : Système Comptable OHADA (Organisation pour l'Harmonisation en Afrique du Droit des Affaires), référentiel comptable utilisé par les pays francophones d'Afrique de l'Ouest et centrale.

TresorIA : nom du chatbot IA de MoneXa, qui répond aux questions des utilisateurs en langage naturel à partir de KPIs pré-calculés, sans jamais générer de SQL.

19.3 Contacts

Équipe MoneXa — ESIG Tech Arena 2026, Défi 2. Email : team.monexa@esig.example. Slack : canal #monexa-defi2 (invitation sur demande). Site de démo : <https://monexa-demo.vercel.app> (en ligne pendant 7 jours post-hackathon). Dépôt Git public : <https://github.com/monexa-esig/monexa-backend> (backend) et <https://github.com/monexa-esig/monexa-app> (Flutter).

Le présent cahier des charges constitue le document de référence du projet MoneXa pour le hackathon ESIG Tech Arena 2026, Défi 2. Il est susceptible d'évolutions mineures pendant l'événement en fonction des retours du partenaire industriel et des mentors. Toute modification significative est tracée dans le dépôt Git et communiquée à l'équipe.